

Merkzettel – k6 Lasttest & Infrastruktur-Konfiguration

Projekt: WebAPI_Net9ASP · **Stand:** Mai 2026

Umgebung: 4 CPU-Kerne · 2 Backend-Instanzen · 1 nginx · 1 MySQL (Docker)

1 · Kapazitätsformel (Little’s Law)

Herleitung

Jede k6-VU führt pro Iteration aus:

```
[Request: T_measured] [sleep: T_sleep] [Request: T_measured] ...
```

Symbol	Bedeutung
V	Anzahl gleichzeitiger VUs (Virtual Users)
T_measured	Gemessene reine Antwortzeit pro Iteration (in Sekunden)
T_sleep	sleep() -Dauer zwischen Iterationen (in Sekunden)
B	Anzahl Backend-Instanzen

Formel – Gleichzeitige Requests pro Backend

$$\text{ConcurrentReq/Backend} = \frac{V \times T_{\text{measured}}}{(T_{\text{measured}} + T_{\text{sleep}}) \times B}$$

Intuition: Von jeder VU-Iteration-Zeit ($T_{\text{measured}} + T_{\text{sleep}}$) belegt nur T_{measured} tatsächlich einen Backend-Slot.
Aufgeteilt auf B Backends ergibt sich die Last pro Instanz.

Formel – MySQL Connection-Pool pro Backend

$\text{MaxPoolSize} = \text{ConcurrentReq/Backend} \times \text{SafetyFactor}$ (Empfehlung: $\times 2$)

Formel – MySQL max_connections (gesamt)

$$\text{max_connections} = B \times \text{MaxPoolSize} \times 1,1$$

2 · Berechnungsbeispiel (unser Setup)

Parameter:

- $T_{\text{measured}} = 0,29 \text{ s}$ (gemessen mit `api-test_avgT_iteration.js`, $p(90) = 290,9 \text{ ms}$)
- $T_{\text{sleep}} = 1 \text{ s}$ (in `api-tests.js`)
- $B = 2$ Backends
- Peak-VUs: $V = 500$ (Dimensionierung), getestet: $V = 400$ (stabiler Betrieb)

Für $V = 500$ (Dimensionierungslast):

$$\text{ConcurrentReq/Backend} = \frac{500 \times 0,29}{(0,29 + 1,0) \times 2} = \frac{145}{2,58} \approx 56$$

$$\text{MaxPoolSize} = 56 \times 2 = 112$$

$$\text{max_connections} = 2 \times 112 \times 1,1 = 246,4 \rightarrow 250$$

Für $V = 400$ (stabiler Betrieb):

$$\text{ConcurrentReq/Backend} = \frac{400 \times 0,29}{1,29 \times 2} = \frac{116}{2,58} \approx 45$$

3 · Timeout-Kette (Strategie A – empfohlen)

Die Timeouts müssen **streng aufsteigend** konfiguriert sein, damit der äußerste Layer zuerst abbricht und alle inneren Layer sauber aufräumen können:

k6 (Client) nginx (Proxy) ASP.NET / MySQL (Backend)
timeout: 10s < proxy_read: 20s < DB Connection Timeout: 25s

Layer	Parameter	Wert
k6	{ timeout: '10s' } pro Request	10 s

nginx	proxy_connect_timeout	10 s
nginx	proxy_read_timeout	20 s
MySQL Pool	Connection Timeout=	25 s
Kestrel	RequestHeadersTimeout	15 s
Kestrel	KeepAliveTimeout	60 s

***Regel:** Wenn zwei Timeouts gleich sind, kann der falsche zuerst feuern immer gestaffelt konfigurieren.*

4 · Konfigurationsdateien – wichtige Stellschrauben

docker-compose.yml – MySQL

```
command: >
--max_connections=250
--innodb_buffer_pool_size=256M
```

SqlServerDatabaseInitializer.cs – Connection-String

```
Max Pool Size=112;
Min Pool Size=5;
Connection Timeout=25;
Connection Lifetime=300;
```

nginx.conf

```
worker_processes auto;    # 1 Worker-Prozess pro CPU-Kern    optimale CPU-Auslastu

events {
    multi_accept on;      # Worker akzeptiert alle wartenden Verbindungen auf einmal
    worker_connections 4096; # Max. gleichzeitige Verbindungen pro Worker-Prozess
                           # Gesamt-Max = worker_processes × worker_connections
}
```

```

upstream webapi_Net9 {
    least_conn;          # Load-Balancing: Request geht an Backend mit wenigsten aktive
                        # (besser als round-robin bei unterschiedlich langen Requests)
    server backend:5100 max_fails=3 fail_timeout=30s;
    keepalive 64;        # Bis zu 64 idle TCP-Verbindungen zu Backends offen halten
                        # spart TCP-Handshake-Overhead bei jedem Request
}

proxy_connect_timeout 10s; # Max. Wartezeit bis TCP-Verbindung zum Backend steht
proxy_read_timeout 20s; # Max. Wartezeit auf Antwortdaten vom Backend (nach Ver
                        # Muss > k6-timeout (10s), damit nginx nicht zuerst abbricht
proxy_http_version 1.1; # HTTP/1.1 aktiviert ermöglicht keepalive zum Backend
proxy_set_header Connection ""; # Entfernt "Connection: close" Header keepalive
proxy_buffering on; # nginx puffert Backend-Antwort vollständig Backend-Verbir
                        # wird früher freigegeben, Client-Übertragung läuft separat
proxy_buffer_size 4k; # Puffer für Antwort-Header
proxy_buffers 8 8k; # 8 Puffer à 8 KB für Antwort-Body (= 64 KB gesamt pro Reque

```

Program.cs – Kestrel / ThreadPool

```

builder.WebHost.ConfigureKestrel(options => {
    // Max. gleichzeitige TCP-Verbindungen die Kestrel offen hält.
    // Anfragen darüber werden in die OS-TCP-Accept-Queue eingereiht.
    // Sollte >= nginx worker_connections sein.
    options.Limits.MaxConcurrentConnections = 1000;

    // Wie lange eine idle Keep-Alive-Verbindung offen bleibt.
    // Zu kurz häufige TCP-Reconnects; zu lang Ressourcen-Verschwendung.
    options.Limits.KeepAliveTimeout = TimeSpan.FromSeconds(60);

    // Max. Zeit, die Kestrel auf den kompletten HTTP-Request-Header wartet.
    // Schutz vor Slow-Header-Angriffen und hängenden Verbindungen.
    // Muss < proxy_read_timeout (20s) sein, damit Kestrel zuerst aufräumt.
    options.Limits.RequestHeadersTimeout = TimeSpan.FromSeconds(15);
});

// Setzt das Minimum an sofort verfügbaren ThreadPool-Threads (Worker, IO).
// Verhindert das "ThreadPool-Starvation"-Problem:

```

```
// Ohne dies startet .NET unter Last Threads nur langsam (1 neuer Thread/500ms),  
// was async/await-Fortsetzungen verzögert und Latenz sprunghaft erhöht.  
ThreadPool.SetMinThreads(100, 100);
```

5 • k6-Testskripte

api-tests.js – Haupttest (CRUD)

```
const p    = { timeout: '10s' };  
const pJson = { timeout: '10s', headers: { 'Content-Type': 'application/json' } };  
const pAuth = { timeout: '10s', headers: { 'Authorization': `Bearer ${token}` } };  
const pAuthJson = { timeout: '10s', headers: {  
  'Authorization': `Bearer ${token}`,  
  'Content-Type': 'application/json'  
}};  
  
// Dynamische ID aus POST-Response  
const id = res4.status === 201  
  ? JSON.parse(res4.body).data.id  
  : null;  
  
// Null-sichere PATCH/DELETE  
const res5 = id ? http.patch(`.../${id}`, patchBody, pAuthJson) : null;  
const res6 = id ? http.del(`.../${id}`, null, pAuth) : null;  
  
sleep(1); // T_sleep = 1s
```

Stages (stabiler Betrieb bis 400 VUs):

```
stages: [  
  { duration: '1m', target: 50 },  
  { duration: '2m', target: 100 },  
  { duration: '3m', target: 200 },  
  { duration: '3m', target: 400 },  
  { duration: '2m', target: 200 },  
  { duration: '1m', target: 0 },  
]
```

Thresholds (Stufe 6):

```
thresholds: {  
  http_req_failed: ['rate < 0.05'], // < 5 % Fehler  
  http_req_duration: ['p(99) < 5000'], // p(99) < 5 s  
}
```

api-test_avgT_iteration.js – T_measured messen

```
import { Trend } from 'k6/metrics';  
const iterDuration = new Trend('custom_iter_duration');  
  
export default function () {  
  const start = Date.now();  
  // ... alle Requests einer Iteration ...  
  iterDuration.add(Date.now() - start);  
  // kein sleep() misst reine Antwortzeit  
}
```

Generiert: Mai 2026 · GitHub Copilot (Claude Sonnet 4.6)